Ľubomír Bojda, 5ZYI21

Dokumentácia – Algoritmy a údajové štruktúry 1 – Semestrálna práca

1. Návrh aplikácie

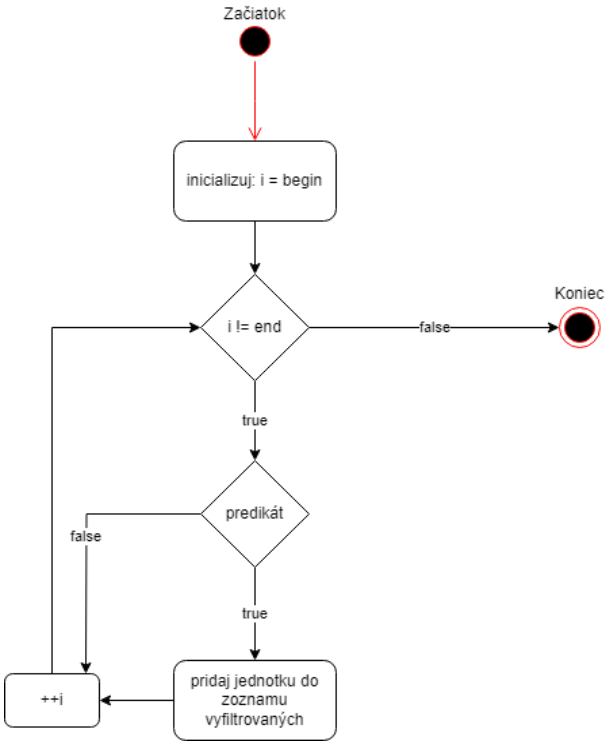
Popis:

Aplikácia slúži na načítanie, spracovanie a následné prehľadávanie údajov o územných jednotkách Českej Republiky.

Funkcionalita:

V aplikácii je možné vyhľadať údaje o územnej jednotke, ktorej typ a názov je priamo zadaný alebo prehľadávať jednotky ČR po úrovniach a v každej úrovni vyhľadať podradené územné jednotky pomocou textu, ktorý obsahuje ich názov alebo pomocou zadaného typu.

UML diagram aktivít algoritmu:



Algoritmus – programátorská príručka

Trieda algoritmus obsahuje metódu vyfiltrujAlgoritmom, ktorá preberá 3 parametre:

- begin – iterátor ukazujúci na prvý prvok v štruktúre
- end - iterátor ukazujúci na posledný prvok v štruktúre
- predikát – funkcia s návratovou hodnotou boolean, ktorú zadáva používateľ podľa toho, čo chce vyfiltrovať

  Následne je možné vyfiltrované jednotky vypísať na konzolu metódou vypisVyfiltrovaneJednotky.

Načítavanie údajov

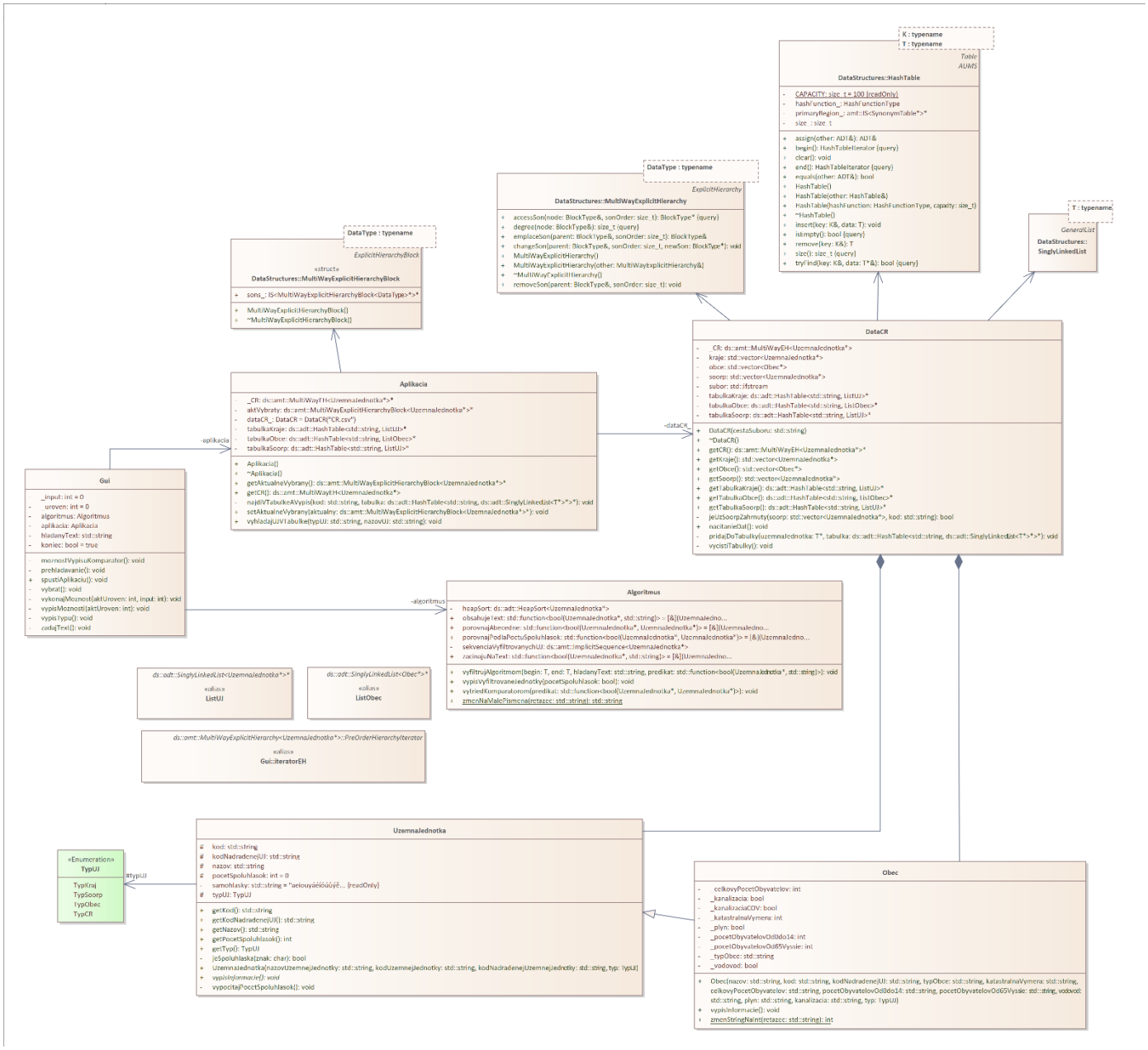
Úprava vstupného súboru:

V súbore „CR.csv“ som upravil prvý stĺpec, ktorý obsahuje kód kraja, v ktorom sa územná jednotka nachádza. Stĺpec bol však prázdny, ak riadok obsahoval názov samotného kraja a teda kód každého kraja sa nachádzal prvý krát až v ďalšom riadku pri podradenej jednotke daného kraja. Preto som kód kraja doplnil aj do riadku s názvom samotného kraja, čo mi uľahčilo načítavanie údajov v programe.

Postup načítavania údajov:

Pri načítavaní údajov v podstate opakujem krok spracovania jedného riadku súboru, pokiaľ vstupný súbor má ďalší riadok. Takže si získam celý riadok a vložím ho do premennej typu string. Tento string si následne rozdelím na viacero stringov (každý ďalší string obsahuje text riadku, kým textom nie je bodkočiarka, ktorá oddeľuje jednotlivé stĺpce), ktoré vložím do vektoru stringov. Takže mám k dispozícii vektor, ktorého každý prvok obsahuje jeden stĺpec riadku. Následne porovnávam prvky vektora tak, aby som vedel, čo obsahuje riadok, ktorý aktuálne obsluhujem. Riadok, ktorý obsahuje kraj má druhý stĺpec prázdny, keďže kraj nemá kód soorp. Takže ak je druhý stĺpec v aktuálnom riadku prázdny, vytvorím novú územnú jednotku, ktorej parametre zadám pomocou prvkov vo vektore stringov. Túto územnú jednotku následne pridám do vektoru krajov (pre úroveň 1), do hierarchie na úroveň krajov (pre úroveň 2) a do tabuľky krajov (pre úroveň 3). Ak však druhý stĺpec v riadku nie je prázdny, jedná sa o riadok, ktorý obsahuje soorp a aj obec, ktorá sa nachádza v danom soorpe. Soorpy sa však v riadkoch opakujú, takže najskôr overím pomocou kódu, či už som daný soorp zaradil. Ak nie, tak vytvorím novú územnú jednotku a pridám ju do vektoru soorpov, hierarchie (ako syna posledného vloženého kraja) a tabuľky soorpov. Potom sa presúvam na pridanie obce. Ak soorp už zaradený je, tak sa nikde nepridáva a pokračuje sa na pridanie obce – vytvorí sa nová obec, pridá sa do vektoru obcí, do hierarchie (ako syna posledného pridaného soorpu) a do tabuľky obcí.

2. UML diagram tried



3. Rozbor vhodnosti použitia zvolených údajových štruktúr

Viaccestná explicitná hierarchia – potrebujeme viaccestnú hierarchiu, pretože nie každá územná jednotka má rovnaký počet synov (teda napr. nie každý kraj ČR má rovnaký počet soorp). Na rozdiel od explicitnej hierarchie, implicitná umožňuje implementovať iba K-cestné hierarchie, teda je potrebné presne určiť počet synov každého vrchola.

Tabuľka s rozptýlenými záznamami (Hash Table) – postavená na myšlienke, že operácia nájdi je analogická k operácii sprístupni v sekvenciách (tá je v implicitných sekvenciách konštantná – veľmi rýchla). Tabuľka s rozptýlenými záznamami využíva implicitnú sekvenciu – všetky operácie realizuje prostredníctvom operácie sprístupni, takže je táto tabuľka teoreticky najrýchlejšou implementáciou tabuľky. Tabuľky však

pracujú s prvkami podľa kľúča, no sekvencie podľa indexu, takže je potrebné kľúč transformovať na celočíselný index (na to slúži hešovacia funkcia).

Jednostranne zreťazený zoznam – v práci slúži na uchovávanie údajov s rovnakými kľúčmi (duplicity). Jednostranne zreťazený zoznam používa explicitnú sekvenciu – operácia vlož posledný je veľmi rýchla (zložitosť O(1)).

4. Používateľská príručka

Po štarte aplikácie sa otvorí konzola, na ktorej sa vypíše menu s 3 možnosťami:

1. Prehľadávať ČR:

   Po zvolení tejto možnosti je možné pohybovať sa po úrovniach územných jednotiek ČR vďaka implementácii hierarchie. Začína sa na najvyššej úrovni (ČR), odtiaľ je možné posunúť sa na nižšiu úroveň vybratím kraja z ponuky. Na úrovni je zasa možnosť vybrať soorp a presunúť sa na nižšiu úroveň. Na jednotlivých úrovniach je možnosť vrátiť sa späť na nadradenú územnú (ak je to možné) a taktiež vykonávať rôzne akcie:

   - vypísať podradené územné jednotky podľa toho či obsahujú alebo začínajú na zadaný text – po výbere užívateľ zadá hľadaný text a následne má na výber z troch možností :
     - iba vypísať územné jednotky;
     - vypísať územné jednotky zoradené abecedne;
     - vypísať územné jednotky zoradené podľa počtu spoluhlások v názve jednotky.
   - vypísať všetky podradené územné jednotky daného typu – po zvolení užívateľ zadá typ jednotky a následne sa územné jednotky vypíšu;
   - vrátiť sa späť do hlavného menu na výber hlavných možností.

2. Vyhľadať informácie o územnej jednotke, ktorej názov a typ sú zadané používateľom:

   Užívateľ aplikácie zadá najskôr typ územnej jednotky a následne jej názov. Ak existuje územná jednotka so zadaným typom a názvom, tak sa vypíšu všetky jej údaje. Ak je takých jednotiek viac, vypíšu sa všetky, ktoré vyhovujú zadanému typu a názvu.

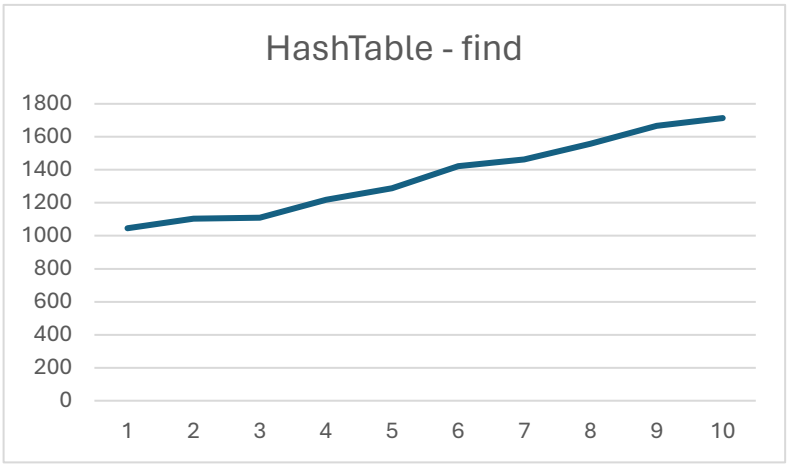
3. Ukončiť aplikáciu:

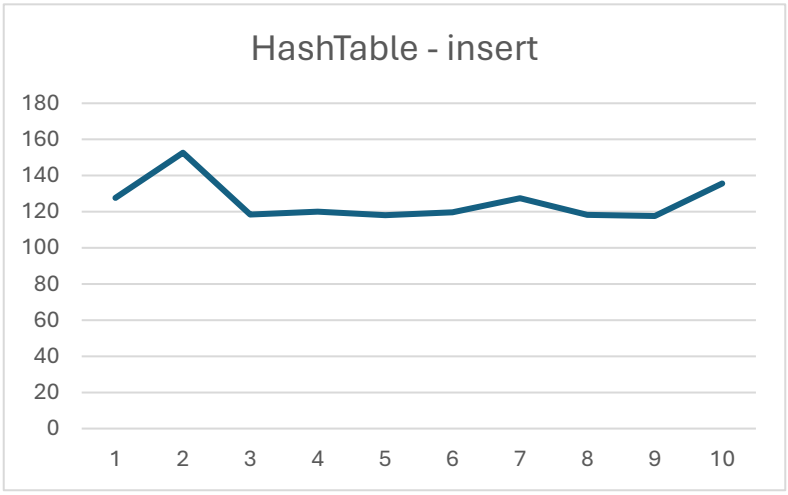
   Aplikácia sa korektne ukončí bez memory leakov (halda je po ukončení čistá).

5. Zoznam použitých štruktúr implementovaných na cvičeniach

- viaccestná explicitná hierarchia;
- tabuľka s rozptýlenými záznamami (HashTable) – naprogramovaná pomocou pseudokódov;
- jednostranne zreťazený zoznam.

6. Analýza časovej zložitosti zvolenej tabuľky





Zložitosť operácie insert pri našej analýze vyzerá O(1), aj keď môžeme vidieť jemné výkyvy, to môže byť spôsobené aj množstvom procesov na pozadí operačného systému.

Zložitosť perácia find pri našej analýzej vyzerá O(n), hoci by operácia mala mať zložitosť O(1). Zrejme sa taktiež môže jednať o množstvo procesov, ktoré bežia na pozadí.